

Developer Setup — MisFinanzas

Requisitos

- Python 3.11+
- pip

No se necesita Node.js, Docker ni ninguna otra herramienta. El frontend se carga desde CDN.

Setup local (5 minutos)

```
# 1. Clonar
git clone https://github.com/lans757/misfinanzas.git
cd misfinanzas

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Crear archivo de configuración
echo "GEMINI_API_KEY=" > .env

# 4. Levantar el servidor
uvicorn app:app --host 0.0.0.0 --port 8000 --reload

# 5. Abrir en el browser
# http://localhost:8000
```

El flag --reload recarga automáticamente el servidor cuando cambiás app.py.

La base de datos SQLite (finanzas.db) se crea automáticamente al primer arranque.

Variables de entorno

Variable	Requerida	Descripción
DATABASE_URL	No (dev)	URL de PostgreSQL. Si no está, usa SQLite.
GEMINI_API_KEY	No	API key de Google Gemini. Se puede configurar desde

En desarrollo, estas variables se leen del archivo .env. En producción, se configuran en el dashboard de Render (o la plataforma que uses).

Archivos importantes

```

misfinanzas/
├─ app.py           # Backend completo (FastAPI + modelos + endpoints)
├─ requirements.txt  # Dependencias Python
├─ render.yaml      # Configuración de despliegue en Render
├─ schema.sql       # Schema SQL de referencia (no se usa al iniciar)
├─ migrate_db.py    # Script one-time para migrar SQLite → PostgreSQL
├─ .env             # Variables de entorno locales (no commitar)
├─ .secret_key      # Clave JWT generada automáticamente (no commitar)
├─ finanzas.db      # Base de datos SQLite local (no commitar)
├─ static/
│   └─ index.html   # Frontend completo (React + Tailwind + Chart.js via CDN)
└─ docs/
    ├─ architecture.md # Este documento + más detalles técnicos
    ├─ user-guide.md   # Guía de uso para usuarios finales
    └─ dev-setup.md    # Esta guía

```

API interactiva

FastAPI genera documentación automática en:

- Swagger UI: <http://localhost:8000/docs>
- ReDoc: <http://localhost:8000/redoc>

Desde Swagger podés probar todos los endpoints sin necesidad de un cliente HTTP externo.

Hacer cambios

Backend (app.py)

El archivo está organizado en secciones claramente marcadas:

```

Constantes y config
Modelos SQLAlchemy (User, Transaction, Goal, BudgetAlert, RateHistory)
Migraciones SQLite runtime
Utilidades de auth (JWT, bcrypt)
Fetchers de tasas externas (BCV, Binance P2P)
Schemas Pydantic
Endpoints FastAPI (auth, rates, transactions, goals, budgets, AI, export)

```

Después de editar, uvicorn recarga automáticamente si levantaste con `--reload`.

Agregar un nuevo modelo:

1. Definir la clase heredando de Base

2. `Base.metadata.create_all(bind=engine)` lo crea al iniciar (ya existe esa línea)
3. Si la tabla ya existe en SQLite y querés agregar una columna: agregar un `ALTER TABLE` en el bloque de migraciones runtime (líneas ~167-178)

Frontend (static/index.html)

Todo el React está en un `<script type="text/babel">`. Editar directamente y refrescar el browser — no hay build.

La estructura de componentes:

```
App (estado global, handlers)
├─ AuthScreen
├─ Header
├─ DashboardTab
├─ TransactionsTab → TransactionForm (crear/editar)
├─ SavingsTab
├─ BudgetTab
└─ AITab
```

El cliente API vive en la función `makeApi(token, onUnauthorized)`. Para agregar un nuevo endpoint, agregar el método ahí primero.

Despliegue en Render

El proyecto incluye `render.yaml` con la configuración lista.

1. Hacer fork/push a GitHub
2. Crear cuenta en `render.com`
3. New → Web Service → conectar el repositorio
4. Render detecta `render.yaml` automáticamente
5. Agregar variables de entorno en el dashboard de Render:
 - `DATABASE_URL`: URL de PostgreSQL (Render ofrece PostgreSQL gratis en plan hobby)
 - `GEMINI_API_KEY`: opcional, se puede configurar desde la UI de la app

Importante: el plan free de Render no incluye disco persistente. Sin `DATABASE_URL`, la base de datos SQLite se pierde en cada redeploy. Usar PostgreSQL en producción.

Migrar datos de SQLite a PostgreSQL

Si tenés datos en SQLite y querés pasarlos a PostgreSQL:

```
# Configurar ambas URLs en el entorno
export OLD_DB="sqlite:///./finanzas.db"
export NEW_DB="postgresql://user:pass@host/dbname"

python migrate_db.py
```

Convenciones de código

- Sin type hints estrictos en el frontend (es JavaScript in-browser)
 - Backend: type hints en todos los parámetros de función pública
 - Nombres de variables en español en el dominio de negocio (monto, saldo, tasa), inglés en infraestructura
 - Schemas Pydantic con sufijo Create, Update, Out para distinguir entradas de salidas
 - Todos los endpoints protegidos usan Depends(get_current_user) — nunca leer el token manualmente
-

Troubleshooting

Error "duplicate column" al iniciar

Normal — la migración runtime intenta agregar columnas que ya existen. El error se ignora deliberadamente.

func.strftime falla en producción

La función month_label(col) en app.py maneja esto. Si ves este error, verificar que todas las queries de filtrado por mes usen month_label() en lugar de func.strftime() directo.

La API key de Gemini no persiste entre reinicios en Render

Porque en producción no se escribe al disco. Configurar GEMINI_API_KEY como variable de entorno en el dashboard de Render.

Binance P2P no responde

El endpoint de Binance bloquea requests de algunos datacenters. En producción puede fallar. La app maneja el error gracefully (devuelve null para binance_p2p).